

Percobaan I
Instruction Fetch dan Instruction Decode

Danny Rahmat Pratama (13224019)
Asisten : Nabila Luna (13223059)
Tanggal Percobaan : 28/10/2026
EL3111 Praktikum Arsitektur Sistem Komputer
Laboratorium Sinyal dan Sistem – Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

***Abstrak*—Percobaan ini merancang empat komponen bagian instruction fetch dan instruction decode pada mikroprosesor RISC-V RV32I single-cycle: instruction memory 32 word × 32-bit, register file 32 × 32-bit dengan dua port baca dan satu port tulis, control unit dengan 16 sinyal kendali, dan data memory 256 word × 32-bit. Sebelum praktikum, dirancang pula multiplexer 2-to-1, PC+4 adder, dan program counter, serta program penjumlahan bilangan 1 sampai 10 diterjemahkan ke assembly RISC-V dan disimulasikan di Venus. Setiap komponen ditulis dalam Verilog HDL, diuji fungsional dengan testbench dan waveform, lalu dianalisis static timing-nya di Quartus untuk memperoleh setup slack dan hold slack. Keempat testbench menghasilkan PASS tanpa error. Instruction memory memiliki setup slack 4,580 ns dan hold slack 2,079 ns dengan frekuensi maksimum 184,5 MHz. Register file memiliki setup slack −2,466 ns sehingga periode clock dinaikkan menjadi 12,466 ns dengan frekuensi maksimum 80,257 MHz. Control unit memenuhi setup dan hold dengan frekuensi maksimum 119,99 MHz, dan keluaran sinyal kendali untuk lima instruksi uji sesuai dengan tugas pendahuluan. Data memory memiliki setup slack −0,812 ns sehingga periode dinaikkan menjadi 10,812 ns dengan frekuensi maksimum 92,49 MHz. Hold slack seluruh komponen bernilai positif.**

***Kata Kunci*—datapath, control, RISC-V RV32I, static timing analysis, Verilog HDL**

I. Pendahuluan

Percobaan I membangun blok-blok yang melakukan pengambilan dan *decoding* instruksi. Tujuan percobaan ini adalah memahami arsitektur dan alur eksekusi RISC-V RV32I *single-cycle*, memahami *instruction set* intinya, dan menjalankan program sederhana pada simulator Venus. Selain itu, praktikan merealisasikan *instruction memory*, *register file*, *control unit*, dan *data memory* dalam Verilog HDL yang dapat disintesis dan diuji secara fungsional maupun *timing*.

Pada tahap pendahuluan, dirancang tiga komponen pendukung: *multiplexer* 2-to-1 32-bit, *PC+4 adder*, dan *program counter*. Dilakukan juga penerjemahan program C penjumlahan 1 sampai 10 ke *assembly* RISC-V dengan instruksi ADDI, ADD, dan BLT, memverifikasi nya di Venus, dan mencatat *machine code*-nya. *Machine code* tersebut menjadi isi *instruction memory*.

Pada praktikum, keempat komponen utama dirancang sebagai berikut:

1. *Instruction memory* menerima alamat *byte* dari PC, tetapi diindeks per *word* dan mengeluarkan NOP saat *reset*.
2. *Register file* menulis pada tepi naik *clock*, membaca pada tepi turun *clock*, dan menjaga register x0 tetap bernilai nol.
3. *Control unit* menghasilkan 16 sinyal kendali dari *opcode*, funct3, dan funct7 instruksi.
4. *Data memory* mendukung penyimpanan *byte*, *halfword*, dan *word* sesuai tipe *store*.

Setiap komponen ditulis dalam Verilog HDL dan diverifikasi dengan *testbench* yang membandingkan keluaran dengan nilai yang diharapkan, dilengkapi pengamatan *waveform*. Analisis *static timing* di Quartus kemudian menghasilkan *setup slack* dan *hold slack*, yang dipakai untuk menentukan frekuensi maksimum tiap komponen.

II. Landasan Teoretis

A. *Arsitektur Instruksi Set RISC-V*

Arsitektur didefinisikan sebagai set instruksi dan lokasi *operand* (memori dan register). Untuk memahami arsitektur komputer, maka diperlukan untuk mengetahui set instruksi dari komputer tersebut. Arsitektur dari RISC-V menggunakan instruksi yang berukuran 32-bit. Pada arsitektur RISC-V bagian yang dioperasikan disebut sebagai *operand* dan terdiri dari:

1. Register

Instruksi membutuhkan akses cepat pada *operand* sehingga operasi dapat berjalan dengan cepat, tetapi

operand yang tersimpan dalam memori membutuhkan waktu yang lambat untuk mengaksesnya. Hal ini menyebabkan arsitektur menggunakan register untuk menyimpan *operand* yang akan digunakan. Dalam arsitektur RISC-V, terdapat 32 register yang disimpan dalam *register file*.

Setiap register pada *register file* memiliki fungsinya masing-masing, fungsi tersebut dapat dilihat pada Tabel 2.1.1.

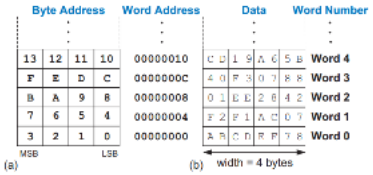
TABEL 2.1.1
REGISTER SET RISC-V

Nama	Register	Fungsi
zero	x0	Nilai konstan 0
ra	x1	*Return address*
sp	x2	*Stack pointer*
gp	x3	*Global pointer*
tp	x4	*Thread pointer*
t0-2	x5-7	*Temporary Register*
s0/fp	x8	*Saved Register/Frame Pointer*
s1	x9	*Saved Register*
a0-1	x10-11	*Function arguments/Return values*
a2-7	x12-17	*Function arguments*
s2-11	x18-27	*Saved registers*
t3-6	x28-31	*Temporary registers*

2. Memori

Jika hanya digunakan register untuk menyimpan *operand*, maka jumlah data yang dapat tersimpan terbatas pada 32 variabel. Oleh karena itu, data juga dapat disimpan dalam memori. Namun berbeda dengan register yang berukuran kecil dan cepat, memori berukuran lebih besar dan lambat. Pada RISC-V, instruksi beroperasi pada register, sehingga data yang tersimpan pada memori harus dipindahkan terlebih dahulu pada memori. Dengan menggunakan kombinasi antara register dan memori, program dapat mengakses data yang cukup besar dengan waktu yang cepat.

RISC-V menggunakan *byte-addressable memory* yang berarti setiap byte pada memori memiliki alamat tersndiri. Kalimat 32-bit terdiri dari 8-bit byte, sehingga setiap alamat dari kalimat adalah kelipatan dari 4.



Gambar 2.1.1 *Byte-addressable memory* RISC-V

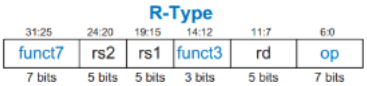
3. Konstan

Instruksi dari RISC-V juga dapat menggunakan konstan atau *immediate operand* yang nilainya diambil langsung dari instruksi tanpa membutuhkan mengakses memori dan register. Konstan menggunakan format 12-bit signed, namun diperlebar menjadi *sign-extended* 32 bit. Konstan dapat digunakan untuk melakukan inisialisasi variabel.

Untuk mempermudah permbacaan, RISC-V memiliki *machine language* yang menjelaskan instruksi set yang digunakan. Instruksi set pada RISC-V memiliki empat buah format utama, yaitu:

1. *R-type Instruction*

R-type atau *register type* menggunakan tiga buah register sebagai *operand* dan satu register sebagai destinasi dari hasilnya. Instruksi set tersebut terdiri dari op, rd, funct3, funct7, rs2, dan rs1. Bagian op (*operation code or opcode*), funct3, dan funct7 (*function fields*) menentukan operasi yang akan dilakukan. Dua buah register *operand* adalah rs1 dan rs2 serta register destinasi yaitu rd.



Gambar 2.1.2 Format Instruksi *R-type*

2. *I-type Instruction*

I-type atau *immediate type* menggunakan dua register sebagai *operand* dan satu konstan sebagai *operand*. Format yang digunakan terdiri dari op, rd, funct3, rs1, dan imm. Bagian op dan funct3 menentukan operasi yang akan dilakukan. Register rs1 adalah register yang digunakan sebagai *operand* dan rd adalah register

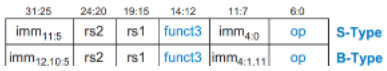
destinasi dari operasi. Bagian imm adalah 12-bit signed konstan yang digunakan sebagai *operand*.



Gambar 2.1.3 Format Instruksi *I-type*

3. *S/B-type Instruction*

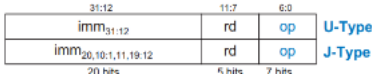
S/B-type atau *store/branch* adalah instruksi yang menggunakan dua buah *operand* dan satu *immidiate operand*. Perbedaannya dengan I-type adalah penggunaan register yang keduanya adalah sebagai sumber. Format yang terdiri atas op, imm, funct3, rs1, dan rs2. Bagian op dan funct3 menentukan operasi yang akan dilakukan. Register rs1 dan rs2 adalah register sumber yang digunakan sebagai *operand*. Perbedaan antara S-type dan B-type adalah bagian imm di *encode*. Pada S-type, bagian tujuh bit teratas imm diletakan pada bit 31:25 dan lima bit terbawah diletakan pada bit 11:7. Sedangkan pada B-type, bit 12 dan bit 10:5 imm diletakan pada bit 31:25 instruksi dan bit 4:1 serta 11 imm diletakan pada bit 11:7 instruksi.



Gambar 2.1.4 Format Instruksi *S/B-type*

4. *U/J-type Instruction*

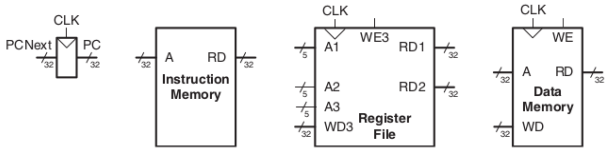
U/J-type atau *upper-immediate/jump* adalah instruksi yang terdiri dari *operand* rd dan 20-bit konstan. Format dari U/J-type terdiri dari op, rd, dan imm. Pada instruksi U-type, imm adalah 20-bit konstan yang merupakan MSB dari 32-bit konstan. Sedangkan pada J-type, imm adalah 20 bit konstan yang merupakan 21-bit dari *jump offset* [1].



Gambar 2.1.5 Format Instruksi *U/J-type*

B. *Mikroarsitektur RISC-V*

Mikroarsiktektur RISC-V dibagi menjadi dua, yaitu *datapath* dan *control unit*. Bagian *datapath* melakukan operasi pada data dan mengandung struktur seperti memori, register, ALU, dan multiplexer. Sedangkan bagian *control unit* menerima instruksi dari *datapath* serta menjelaskan bagaimana *datapath* dapat menjalankan instruksi tersebut. Bagian-bagian yang terdapat pada *control unit* adalah *multiplexer select, register enable,* dan *memory write* untuk mengontrol operasi *datapath*. Pada bagian paling dasarnya, RISC-V mikroarsitektur terdiri dari *program counter, register counter, instruction memory*, dan *data memory*.



Gambar 2.2.1 Bagian dasar dari prosesor RISC-V

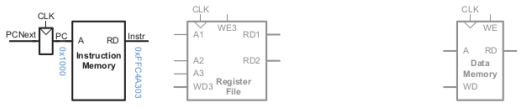
Bagian *program counter* menunjuk pada instruksi saat ini dan saat menerima input, *PCNext*, akan menunjuk pada instruksi setelahnya. *Instruction memory* menerima input 32-bit alamat instruksi dan mengeluarkan 32-bit data instruksi dari alamat tersebut. Bagian register 32-elemen x 32-bit menyimpan register x0-x31, dimana A1 dan A2 menerima register yang akan digunakan, A3 digunakan untuk input alamat, WD3 digunakan untuk melakukan *writing* pada register, dan RD1 serta RD2 digunakan untuk mengeluarkan nilai register 32-bit yang akan digunakan. *Data memory* memiliki satu input untuk *writing* pada data saat *rising edge* dan saat WE bernilai 0 maka akan membaca alamat A dan mengeluarkannya pada output RD.

Mikroarsitektur *single-cycle* berarti seluruh instruksi dieksekusi dalam satu siklus clock. Namun *cycle-time* terbatas oleh instruksi yang paling lambat. Pada mikroarsitektur *multi-cycle*, eksekusi instruksi dilakukan pada beberapa *cycle* yang lebih singkat. Arsitektur ini juga memiliki kelebihan pada *hardware* yang dapat digunakan berulang sehingga mengurangi jumlah kebutuhan seperti adder dan memori. Sedangkan pada mikroarsitektur *pipelined* digunakan *pipelining* untuk *single-cycle* mikroarsitektur sehingga dapat melakukan beberapa instruksi dalam satu *clock*. *Pipelining* membutuhkan penambahan logic untuk menjalankan beberapa instruksi sekaligus sehingga menambahkan harga *hardware*.

1. *Single Cycle Datapath RISC-V*

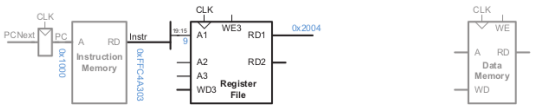
Prosedur pembuatan dalam *datapath single cycle* RISC-V adalah:

a. Bagian *program counter* memiliki alamat instruksi yang akan di eksekusi, sehingga akan dihubungkan dengan input dari *instruction memory*.



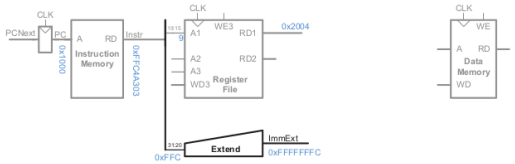
Gambar 2.2.2 *Fetch Instruction* dari Memori

b. Pada instruksi *load word*, dibutuhkan pembacaan sumber register dengan alamat sendiri. Bagian bit instruksi 19 – 15 dihubungkan dengan alamat input A1 pada *register file*.



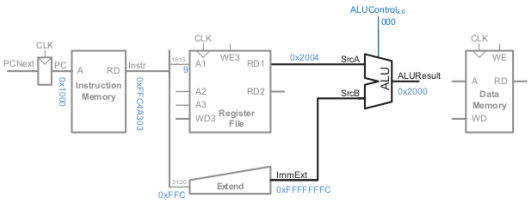
Gambar 2.2.3 Membaca *operand* sumber dari *register file*

c. Selain itu, *lw* juga membutuhkan *offset* yang merupakan bagian bit instruksi 31 – 20 yang akan dilakukan



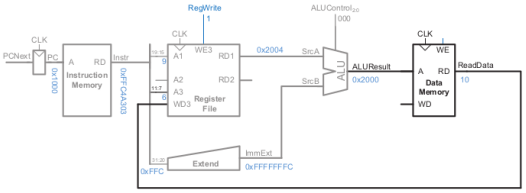
Gambar 2.2.4 *Sign-extend* nilai *immediate*

d. Bagian ALU dihubungkan dengan *operand* yang diterima dari *register file* dan *immediate* setelah dilakukan *sign-extend*.



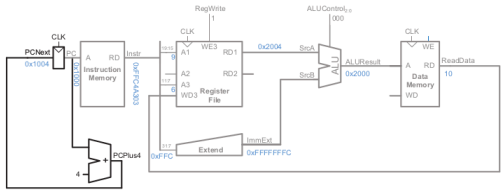
Gambar 2.2.5 Melakukan operasi pada alamat memori

e. Hasil dari ALU mengeluarkan alamat yang dihubungkan dengan bagian *read* alamat dari *data memory* yang dihubungkan kembali dengan *register file* untuk dilakukan pembacaan. Selain itu input A3 pada *register file* juga menerima bit 11 – 7 instruksi yang merupakan destinasi dari *register*.



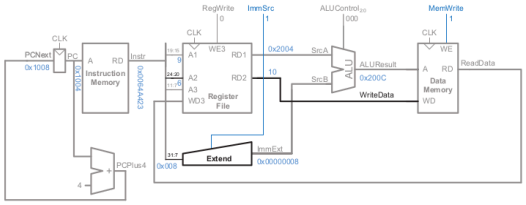
Gambar 2.2.6 Membaca dan menulis hasil kembali pada *register file*

f. Selama instruksi dieksekusi, prosesor juga harus mengambil instruksi setelahnya yang dilakukan dengan melakukan *increment sebesar* 4 bit.



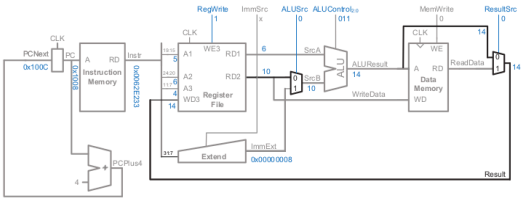
Gambar 2.2.7 *Increment Program Counter*

g. Untuk mengakomodasi instruksi *sw*, *extend* membaca instruksi bit 31 – 7 yang akan digunakan sebagai *immediate*. Selain itu *sw* juga membaca alamat dari register *operand* kedua pada bit 24 – 20 instruksi dan mengeluarkannya pada RD2 dan menulisnya pada *data memory*.



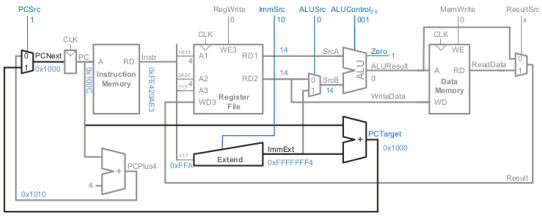
Gambar 2.2.8 Menulis data untuk instruksi sw

h. Selanjutnya untuk mengakomodasi instruksi R-type, ditambahkan mux sebelum input di ALU yang memilih input dari *immediate* atau output register RD2. Selain itu ditambahkan juga mux setelah *data memory* untuk menulis kembali ALU pada *register file*.



Gambar 2.2.9 *Datapath* untuk instruksi R-type

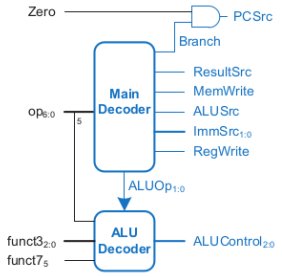
i. Selanjutnya untuk mengakomodasi B-type, ditambahkan *adder* untuk menambahkan *offset immediate* dengan alamat instruksi awal sehingga dapat dilakukan *jump*.



Gambar 2.2.10 *Datapath* untuk instruksi B-type

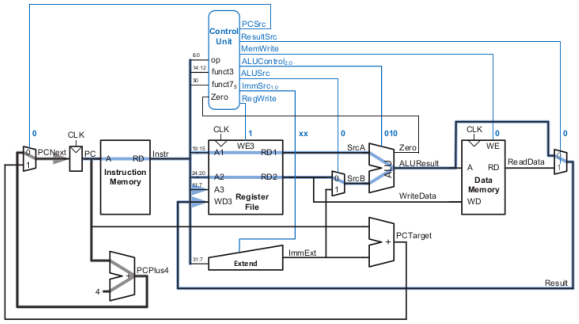
2. *Single Cycle Control Path RISC-V*

Untuk menjalankan seluruh *datapath* tersebut dibutuhkan sinyal kontrol yang berdasarkan instruksi op, funct3, dan funct7. Unit kontrol digunakan untuk melakukan *decode* pada instruksi dan memberikan kontrol sinyal untuk operasi yang harus dilakukan. Bentuk *control unit dapat* dilihat pada Gambar 2.2.11.



Gambar 2.2.11 *Single-cycle Control Unit*

Bagian *control unit* kemudian dihubungkan dengan *datapath* untuk mengontrol blok-blok seperti ALU, Mux, *data memory, register file,* dan *adder*. Bentuk penuh dari mikroarsitektur *single-cycle* dapat dilihat pada Gambar 2.2.12 [1].

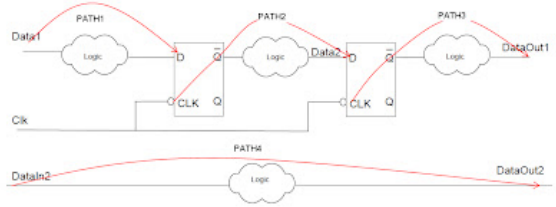


Gambar 2.2.12 Bentuk Penuh *Single-cycle* processor

C. *Static Timing Analysis*

Static Timing Analysis (STA) adalah metode yang digunakan untuk melakukan validasi terhadap performa desain dengan melakukan pengecekan pada seluruh jalur yang memungkinkan untuk terjadinya pelanggaran *timing* untuk kasus paling buruk. STA melihat seluruh delay terburuk untuk setiap elemen logika, tetapi bukan operasi dari rangkaian itu sendiri. Alat STA melakukan analisis untuk semua jalur dari setiap *startpoint* ke seluruh *endpoint* dan membandingkannya dengan *constraint* yang ada pada jalur tersebut. Seluruh jalur harus memiliki *constraint*, sebagian besar jalur memiliki *constraint* akibat perioda dari clock yang digunakan. Dua konsep penting yang perlu diperhatikan adalah *data path* dan *critical path*.

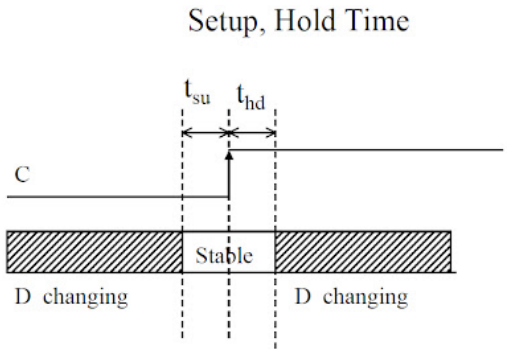
Data path adalah jalur yang *start point* nya berasal dari port input atau pin clock dari desain dan berakhir pada data input pada flip-flop/latch/memory atau output port dari desain.



Gambar 2.3.1 Datapath dari suatu Desain

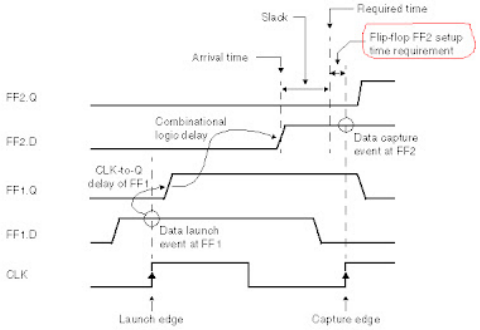
Sedangkan *critical path* adalah jalur yang membentuk delay paling panjang. Jika ditambahkan *gate* pada jalur ini maka akan menambahkan *delay* dari *critical path* yang dapat menghasilkan *timing* yang tidak sesuai (pelanggaran timing).

Selain itu, terdapat konsep *setup* dan *hold* yang digunakan dalam STA. *Setup time* adalah waktu minimum untuk sinyal data harus stabil sebelum *event clock*, sehingga data dapat di sampling dengan stabil. Sedangkan *hold time* adalah waktu minimum sinyal data harus stabil setelah *clock event* sehingga data dapat di sampling dengan stabil.



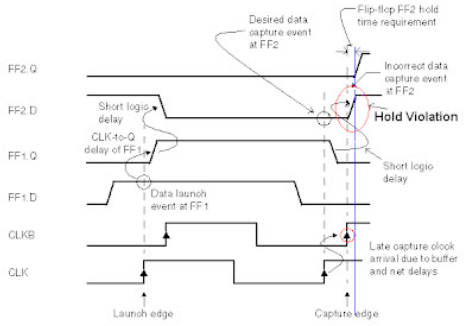
Gambar 2.3.2 *Setup and Hold Time*

Untuk mengecek pelanggaran pada *setup* dan *hold* dapat dilakukan pengecekan dengan menggunakan t_{su} yang merupakan waktu *setup* dan t_{hd}. Jika data belum t_{su} sebelum *clock event*, maka terdapat pelanggaran *setup*.



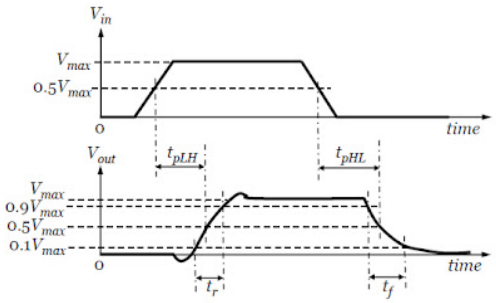
Gambar 2.3.3 *Setup Timing*

Sedangkan untuk mengecek pelanggaran pada *hold*, jika data tidak ditahan secara stabil (terdapat perubahan pada nilai dari data) selama t_{hd} setelah *event clock* maka terdapat pelanggaran *hold*.



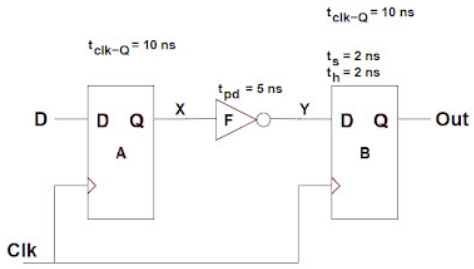
Gambar 2.3.4 *Hold Timing*

Pada rangkaian kombinasional, terdapat *delay propagation* antara input dengan output dari rangkaian tersebut. Delay tersebut adalah waktu yang dibutuhkan dari 50% nilai input hingga mencapai 50% nilai output. Pada Gambar 2.3.5, nilai *delay propagation* adalah t_{pLH} dan t_{pHL} yang merupakan delay dari *rising time* dan *falling time*.



Gambar 2.3.5 *Propagation Delay*

Jika digunakan model pada Gambar 2.3.6, dimana t_{clk-Q} waktu untuk mengeluarkan nilai D pada Q.

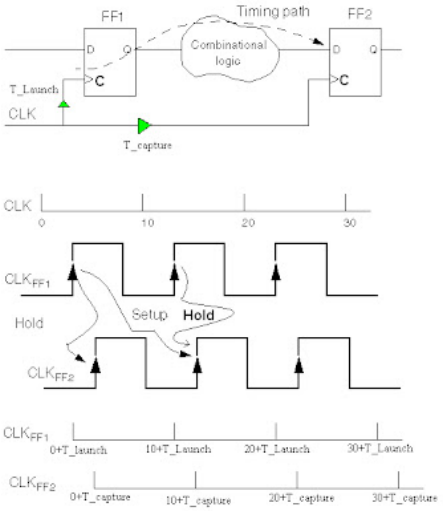


Gambar 2.3.6 *Setup Rangkaian Sekuensial*

Dengan menggunakan nilai dari *setup time* (t_s) dan nilai dari *propagation delay* (t_{pd}), dapat ditentukan nilai dari *slack* dari setup. Nilai *slack setup* adalah waktu yang dimiliki kosong yang dimiliki dalam satu perioda clock (t_{clk}). Nilai dari *slack setup* adalah $t_{setup_slack} = t_{clk} - t_s - t_{pd} - t_{clk-Q}$. Nilai ini dapat digunakan untuk menentukan frekuensi maksimal dari *clock* yang digunakan, yaitu $f_{max} = \frac{1}{t_{clk} - tsetup_slack}$. Terdapat juga *hold slack* yang merupakan waktu yang dimiliki sebelum terdapat perubahan data selanjutnya, waktu *hold slack* dapat dihitung dengan menggunakan $t_{hold_slack} = t_h + t_{pd} + t_{clk-Q}$.

Untuk memperbaiki pelanggaran *setup* dan *delay* terdapat beberapa cara yang dapat digunakan untuk memperbaikinya, yaitu:

1. Jika waktu *hold* membutuhkan waktu lebih lama dibandingkan *combinational logic delay*, maka dapat menambahkan *combinational logic delay* sehingga $t_{dp} + t_{clk-Q} \geq t_h$.
2. Jika waktu *setup* membutuhkan waktu lebih lama dibandingkan pengurangan clock dengan *combinational logic delay*, maka dapat mengurangi *combinational logic delay* sehingga $t_{clock} - t_{dp} - t_{clk-Q} \leq t_s$
3. Jika waktu total *hold* dan *setup* membutuhkan waktu lebih lama dibandingkan clock, maka dapat ditambah waktu clock sehingga $t_{clock} \geq t_h + t_s$
4. Jika *combinational logic delay* tidak dapat diubah, maka dapat ditambahkan *delay* pada clock sehingga $t_{clock} + t_{T_capture} + t_{T_launch} \geq t_h + t_s$ [2]



Gambar 2.3.7 *Setup Rangkaian Sekuensial* dengan delay pada Clock

III. HASIL DAN ANALISIS

A. Tugas I : Perancangan Instruction Memory (RV32I)

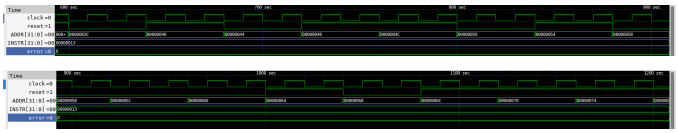
1) Hasil Percobaan

TABEL 3.1.1
HASIL TESTBENCH FUNGSIONAL

```
PASS (ADDRESS = 00000000, ACTUAL = 00100293, EXPECTED = 00100293, RESET = 0)
PASS (ADDRESS = 00000004, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000008, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 0000000C, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000010, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000014, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000018, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 0000001C, ACTUAL = 00000513, EXPECTED = 00000513, RESET = 0)
PASS (ADDRESS = 00000020, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000024, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000028, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 0)
PASS (ADDRESS = 0000002C, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000030, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000034, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 0)
PASS (ADDRESS = 00000038, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 0000003C, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 0)
PASS (ADDRESS = 00000040, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000044, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 0)
PASS (ADDRESS = 00000048, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 0000004C, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000050, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 0)
PASS (ADDRESS = 00000054, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000058, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 0)
PASS (ADDRESS = 0000005C, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 0)
PASS (ADDRESS = 00000060, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 0)
PASS (ADDRESS = 00000064, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000068, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 0)
PASS (ADDRESS = 0000006C, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000070, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000074, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 00000078, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 1)
PASS (ADDRESS = 0000007C, ACTUAL = 00000013, EXPECTED = 00000013, RESET = 0)
TESTBENCH ENDED WITH 0 ERRORS
```







Gambar 3.1.1 *Hasil Waveform Pengujian Fungsional*

	Clock	Slack	End Point TNS
1	virtual_clk	0.506	0.000
2	clock	4.580	0.000

Gambar 3.1.2 *Hasil Analisis Waktu Setup*

	Clock	Slack	End Point TNS
1	clock	2.079	0.000
2	virtual_clk	5.866	0.000

Gambar 3.1.3 *Hasil Analisis Waktu Hold*

Untuk kode *testbench* dapat dilihat pada Lampiran 1.

2) Analisis

Dapat dilihat dari hasil percobaan bahwa analisis fungsional telah lulus tanpa ada *error* pada *testbench*. Dari testbench dimasukan nilai khusus untuk *instruction memory* pada word 1 – 7 dan pada hasil *test bench* terlihat bahwa saat reset bernilai nol mengeluarkan nilai yang sesuai. Pada saat nilai reset adalah satu atau diluar rentang *instruction memory* word 1 – 7, mengeluarkan nilai NOP.

Dari nilai analisis timing, dapat dilihat bahwa nilai masih terdapat *setup slack* dan *hold slack* yang berarti tidak terdapat pelanggaran terhadap *setup* dan *hold*. Terlihat bahwa terdapat delay sebesar $t_{dp} + t_{clk-Q} = 10 - 4,58 = 5,42ns$. Dengan menggunakan persamaan pada bagian *Static Time Analysis* di studi literatur didapatkan:

$$f_{max} = \frac{1}{(10 - 4.580) \times 10^{-9}}$$

$$f_{max} = 184.5MHz$$

3) Diskusi

Dari studi literatur bagian Mikroarsitektur RISC-V, didapatkan bahwa *instruction memory* digunakan untuk mengeluarkan 32-bit instruksi yang berada di dalam memori tersebut. Input dari *instruction memory* sendiri terdapat *byte* alamat dari instruksi-instruksi tersebut yang dilakukan indexing dengan menggunakan *program counter*. Instruksi-instruksi yang disimpan menggunakan set instruksi RISC-V yang terdiri dari opcode, funct3, funct7, rd, rs1, rs2, atau imm. Instruksi tersebut akan selalu berada pada bit yang sama akibat desain dari RISC-V yang membuat *decoding* dari instruksi menjadi lebih mudah dan cepat.

Pada kode *testbench* yang ada pada lampiran 1, digunakan *byte-indexing* yang mengalikan nilai indexer dengan empat untuk mendapatkan *4 byte* setelahnya. Namun dapat dilihat pada kode module sendiri, digunakan *word-indexing* untuk mengakses memori dengan mengubahnya menjadi rentang 4 bit. Secara signifikansi, hasil yang didapatkan adalah sama yaitu mengakses instruksi yang terdapat dalam *instruction memory* yang terpisah secara 4-*byte* atau 1 word (32-bit) yang keduanya berukuran sama.

TABEL 3.1.2
CUPLIKAN KODE VERILOG INSTRUCTION SET

```
// 32 WORD x 32-BIT
 REG [31:0] MEM [0:31];
 // WORD INDEX = PC[6:2] (BIT [1:0] = 2'B00)
 WIRE [4:0] WADDR = ADDR[6:2];
 INTEGER I;


 INITIAL BEGIN
    FOR (I = 0; I < 32; I = I + 1) MEM[I] = 32'H00000013;
    MEM[0] = 32'H00100293;
    MEM[1] = 32'H00000313;
    MEM[2] = 32'H00A00393;
    MEM[3] = 32'H0053C863;
    MEM[4] = 32'H00530333;
    MEM[5] = 32'H00128293;
    MEM[6] = 32'HFF5FF06F;
    MEM[7] = 32'H00000513;
 END
```

B. *Tugas 2 : Perancangan Register File*

1) Hasil Percobaan

TABEL 3.2.1
HASIL TESTBENCH FUNGSIONAL

```
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
TESTBENCH SELESAI DENGAN        0 ERROR
```



Gambar 3.2.1 *Hasil Waveform Pengujian Fungsional*

	Clock	Slack	End Point TNS
1	clock	-2.466	-107.721
2	virtual_clk	-1.560	-93.643

Gambar 3.2.2 *Hasil Analisis Waktu Setup*

	Clock	Slack	End Point TNS
1	clock	1.727	0.000
2	virtual_clk	11.089	0.000

Gambar 3.2.3 *Hasil Analisis Waktu Hold*

Kode untuk *testbench* dapat dilihat pada lampiran 2.

2) Analisis

Dapat dilihat dari hasil percobaan bahwa analisis fungsional telah lulus tanpa ada *error* pada *testbench*. Testbench pertama melakukan percobaan untuk melakukan penulisan pada register x0 – x5, kemudian melakukan pembacaan. Dari hasil *testbench*, terlihat bahwa tidak terdapat perubahan pada register x0 dan penulisan yang dilakukan tidak berpengaruh dengan nilai dari x0 masih 0. Pembacaan pada register x1 – x5 berjalan dengan lancar dan sesuai dengan penulisan yang dilakukan sebelumnya.

Pada *static timing analysis*, dapat dilihat bahwa nilai *setup slack* bernilai negatif, sehingga terdapat pelanggaran *setup*. Untuk memperbaiki hal ini, dapat dilakukan kenaikan perioda clock menjadi $t_{clock} = 10 + 2,466 = 12,466ns$. Sehingga frekuensi maksimalnya $f_{max} = \frac{1}{12,466n} = 80,257MHz$. Terlihat juga bahwa *hold slack* masih bernilai positif sehingga tidak terdapat pelanggaran pada *hold*.

3) Diskusi

Dari studi literatur bagian Mikroarsitektur RISC-V, didapatkan bahwa *register file* digunakan untuk menyimpan 32 x 32-bit register yang berguna untuk menyimpan data dari memori untuk dilakukan operasi di ALU. Register yang dikeluarkan dari *register file* menyesuaikan instruksi yang diterima dari *instruction memory*. Instruksi yang diterima oleh *register file* adalah *operand* rs1, rs2, dan rd. Instruksi tersebut dipecah menjadi bit 19 – 15 untuk *operand* rs1, bit 24 – 20 untuk *operand* rs2, dan bit 11 – 7 untuk rd. Instruksi tersebut akan di *decode* menjadi alamat dari register yang akan digunakan.

Hal yang menjadi menarik adalah register x0 yang selalu bernilai nol, hal ini dikarenakan nilai nol sering digunakan dalam melakukan operasi sehingga menyimpan nilai nol menjadi penting untuk membuat instruksi menjadi tetap *simple*.

Alasan terdapat *register file* adalah untuk akses terhadap data yang lebih cepat, meskipun hanya dapat menyimpan 32 data. Dibandingkan dengan menggunakan *data memory* secara langsung, akses menggunakan *register file* lebih cepat dibandingkan dengan akses menggunakan *data memory*.

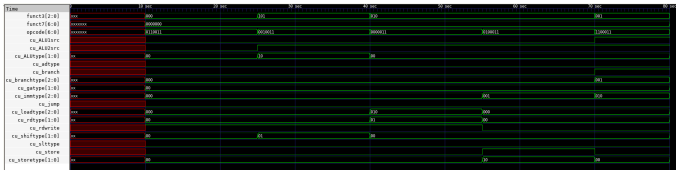
Register file juga melakukan penulisan pada *positive edge clock event* dan pembacaan pada *negative edge clock event*. Hal dilakukan agar terdapat delay combinational $t_{clk-Q} + t_{dp}$ sehingga pembacaan pada menunggu output stabil terlebih dahulu dan dilakukan di *negative edge clock event*.

C. Tugas 3 : Control Unit (CU)

1) Hasil Percobaan

TABEL 3.3.1
HASIL TESTBENCH FUNGSIONAL

```
----------------------------------------------
T=15 | OPCODE=0110011 FUNCT3=000 FUNCT7=0000000
ALU1SRC=0  ALU2SRC=0  IMMTYPE=000
ALUTYPE=00  ADTYPE=0  GATYPE=00  SHIFTYPE=00  SLTTYPE=0
RDTYPE=00  RDWRITE=1  LOADTYPE=000
STORE=0  STORETYPE=00
BRANCH=0  BRANCHTYPE=000  JUMP=0
----------------------------------------------
T=30 | OPCODE=0010011 FUNCT3=101 FUNCT7=0000000
ALU1SRC=0  ALU2SRC=1  IMMTYPE=000
ALUTYPE=10  ADTYPE=0  GATYPE=00  SHIFTYPE=01  SLTTYPE=0
RDTYPE=00  RDWRITE=1  LOADTYPE=000
STORE=0  STORETYPE=00
BRANCH=0  BRANCHTYPE=000  JUMP=0
----------------------------------------------
T=45 | OPCODE=0000011 FUNCT3=010 FUNCT7=0000000
ALU1SRC=0  ALU2SRC=1  IMMTYPE=000
ALUTYPE=00  ADTYPE=0  GATYPE=00  SHIFTYPE=00  SLTTYPE=0
RDTYPE=01  RDWRITE=1  LOADTYPE=010
STORE=0  STORETYPE=00
BRANCH=0  BRANCHTYPE=000  JUMP=0
----------------------------------------------
T=60 | OPCODE=0100011 FUNCT3=010 FUNCT7=0000000
ALU1SRC=0  ALU2SRC=1  IMMTYPE=001
ALUTYPE=00  ADTYPE=0  GATYPE=00  SHIFTYPE=00  SLTTYPE=0
RDTYPE=00  RDWRITE=0  LOADTYPE=000
STORE=1  STORETYPE=10
BRANCH=0  BRANCHTYPE=000  JUMP=0
----------------------------------------------
T=75 | OPCODE=1100011 FUNCT3=001 FUNCT7=0000000
ALU1SRC=1  ALU2SRC=1  IMMTYPE=010
ALUTYPE=00  ADTYPE=0  GATYPE=00  SHIFTYPE=00  SLTTYPE=0
RDTYPE=00  RDWRITE=0  LOADTYPE=000
STORE=0  STORETYPE=00
```



Gambar 3.3.1 *Hasil Waveform Pengujian Fungsional*

	Clock	Slack	End Point TNS
1	virtual_clk	1.666	0.000

Gambar 3.3.2 *Hasil Analisis Waktu Setup*

	Clock	Slack	End Point TNS
1	virtual_clk	5.596	0.000

Gambar 3.3.3 *Hasil Analisis Waktu Hold*

Kode *testbench* dapat dilihat pada lampiran 3.

2) Analisis

Dapat dilihat dari hasil *testbench* bahwa nilai dari output yang diinginkan sudah sesuai jawaban pada tugas pendahuluan.

TABEL 3.3.2
CUPLIKAN OUTPUT YANG DIINGINKAN DARI TUGAS PENDAHULUAN

Instruksi Set	cu_ALU1src	cu_ALU2src	cu_imm type	cu_ALU type	cu_adt ype	cu_gaty pe	cu_shif type	cu_sltty pe	cu_rdty pe	cu_rdw rite	cu_load type	cu_stor e	cu_store type	cu_bra nch	cu_bbr anchtype	cu_jump
ADD x5 x5 x6	0	0	–	00	0	–	–	-	00	1	–	0	–	0	–	0
SRLI x6 x5 5	0	1	000	10	-	–	01	-	00	1	—	0	–	0	–	0
LW x6 12(x2)	0	1	000	00	0	–	–	-	01	1	010	0	–	0	—	0
SW x5 8(x2)	0	1	001	00	0	–	–	-	–	0	—	1	10	0	–	0
BNE x5 x6 4	1	1	010	00	0	–	–	-	–	0	–	0	–	1	001	0

Untuk *static timing analysis*, dapat dilihat bahwa kedua *slack* bernilai positif, sehingga tidak ada pelanggaran *setup* ataupun *hold*. Nilai frekuensi maksimal dapat dihitung

$$f_{max} = \frac{1}{(10 - 1,666)n} = 119,99MHz$$

.

3) Diskusi

Dari studi literatur bagian Mikroarsitektur RISC-V, dapat dilihat bahwa *control unit* digunakan untuk mengatur *datapath* dari mikroarsitektur RISC-V. Komponen-komponen yang diatur oleh *control unit* adalah ALU, Mux, *data memory, register file,* dan *adder*. Untuk melakukan kontrol tersebut, *control unit* menerima instruksi dari *instruction memory* yang kemudian melakukan *decode* terhadap nilai dari *opcode*, *funct3*, dan *funct7*. Bagian-bagian tersebut diubah menjadi sinyal yang digunakan untuk mengontrol *datapath*.

Nilai-nilai dari *opcode*, *funct3*, dan *funct7* adalah unik untuk setiap instruksi sehingga *control unit* dapat membedakan setiap instruksi yang diterima. Instruksi dipecah menjadi bit 6 – 0 untuk opcode, bit 14 – 12 untuk *funct3*, bit 31 – 25 untuk *funct7*. Bergantung pada instruksi tersebut akan dikeluarkan sinyal berdasarkan *instruction set* yang ada pada bagian bagian studi literatur Arsitektur Set Instruksi RISC-V.

D. Tugas 4: Perancangan Data Memory

1) Hasil Percobaan

TABEL 3.4.1
HASIL TESTBENCH FUNGSIONAL

```
PASS
PASS
PASS
PASS
PASS
PASS
PASS
PASS
PASS
PASS
```



Gambar 3.4.1 *Hasil Waveform Pengujian Fungsional*

	Clock	Slack	End Point TNS
1	virtual_clk	-10.819	-304.614
2	clock	-0.812	-134.597

Gambar 3.3.2 *Hasil Analisis Waktu Setup*

	Clock	Slack	End Point TNS
1	clock	3.072	0.000
2	virtual_clk	6.120	0.000

Gambar 3.4.3 *Hasil Analisis Waktu Hold*

Kode *testbench* dapat dilihat pada lampiran 4.

2) Analisis

Dapat dilihat dari hasil *testbench* bahwa nilai dari output sudah sesuai dengan pengujian *testbench*. Pengujian dilakukan dengan memasukkan nilai *random* untuk input rs2, alamat dmem_addr, dan sinyal cu_storetype. Kemudian dilakukan validasi dengan menguji *storetype* yang digunakan dan hasil telah sama dengan validasi sehingga mengeluarkan nilai PASS.

Terlihat bahwa *setup slack* masih bernilai negatif, sehingga terdapat pelanggaran pada *setup*. Untuk memperbaikinya dapat menaikkan perioda *clock* menjadi $t_{clock} = 10 + 0,812 = 10,812ns$. Sehingga nilai dari frekuensi maksimalnya adalah

$$f_{max} = \frac{1}{10,812n} = 92,48982MHz$$

. Terlihat juga bahwa tidak terdapat pelanggaran pada *hold*, karena nilai dari *slack* masih positif.

3) Diskusi

Dari studi literatur bagian Mikroarsitektur RISC-V, dapat dilihat bahwa *data memory* digunakan untuk menyimpan data yang telah dioperasikan atau mengambil data untuk dioperasikan. Perbedaannya dengan *register file* adalah kemampuan menyimpan data yang lebih besar dibandingkan *register*, tetapi dengan waktu pengambilan data yang lebih lambat. Perlu diperhatikan bahwa *data memory* melakukan penulisan pada memori secara sekuensial, tetapi pembacaan secara kombinasional.

E. Analisis Tambahan

1) Tahap-Tahap Eksekusi Single-Cylce RISC-V

Terdapat lima tahap dalam eksekusi instruksi pada *single-cycle* RISC-V, yaitu:

1. Instruction Fetch (IF)

Tahap IF berfungsi mengatur aliran instruksi yang akan diolah pada tahap berikutnya. Instruksi yang sedang dijalankan merupakan instruksi yang berasal dan disimpan dari memory. Pada tahap ini akan diambil 32-bit instruksi dari memori dan melakukan increment terhadap PC sebesar 4 untuk menunjuk pada instruksi setelahnya (pengambilan address memori per byte).

2. Instruction Decode (ID)

Pada tahap ini, instruksi dengan lebar 32-bit akan dipecah sesuai format instruksi yang digunakan. Secara umum, instruksi akan dipecah menjadi bagian alamat destinasi (rd), alamat register sumber (rs1 dan rs2), dan opcode atau kode yang menandakan operasi yang akan dijalankan.

3. Execute / Address Calculation (EX)

Pada tahap ini sebagian besar operasi aritmatika dan logika dilakukan pada Arithmetic and Logical Unit (ALU), misalnya penjumlahan untuk instruksi add. Pada tahap ini juga terdapat tempat untuk meneruskan alamat register kembali ke tahap ID sebagai deteksi hazard untuk pipeline implementation.

4. Data Memory (MEM)

Pada tahap ini, data disimpan dari data memory. Data memory hanya dapat disimpan atau dibaca jika sinyal MemRead dan/atau MemWrite yang sesuai sehingga operasi baca dan/atau tulis pada data memory dapat dilakukan.

5. Write Back (WB)

Tahap ini digunakan untuk mengalirkan data dari data memory atau hasil memory atau hasil perhitungan ALU ke register untuk disimpan dan dipakai untuk instruksi-instruksi berikutnya.

2) Perbedaan RISC-V Generik dengan RISC-V Single-Cylce

Dari studi literatur Mikroaristektur RISC-V, dapat dilihat bahwa karena desain dari generik RISC-V yang menggunakan desain *pipelined* maka hasil *throughput* yang dikeluarkan olehnya lebih besar (dapat mencapai 5 kali karena terdapat 5-stage) dibandingkan dengan *single-cycle*. Namun implementasi dari *pipeline* tersebut membutuhkan hardware yang lebih banyak dibandingkan dengan *single-cyle* yang membuatnya juga memiliki delay yang lebih banyak sehingga dapat menyebabkan penurunan *clock speed*.

IV. Simpulan

- Praktikan telah memahami arsitektur mikroprosesor RISC-V yang terdiri dari *datapath* dan *control*, serta memahami tahap eksekusinya yang terdiri dari *instruction fetch, instruction decode, execute, data memory,* dan *write back*.
- Praktikan telah memahami *instruction set* inti dari RV32i yang terdiri dari instruksi R-type, I-type, S/B-type, dan J/U-type serta melakukan simulasi sederhana di assembler Venus.
- Praktikan telah memahami cara menggunakan Venus dan melakukan simulasi eksekusi program dan setiap instruksi yang digunakannya.
- Praktikan telah membuat *instruction memory, register file, control unit,* dan *data memory* dengan menggunakan Verilog dan melakukan analisis fungsional dengan menggunakan *testbench* dan *waveform* serta melakukan *static timing analysis* dengan menggunakan Quartus untuk mendapatkan nilai *setup time* dan *hold time*.
- Praktikan telah membuat *multiplexer 2-to-1,* PC+4 *adder,* dan *program counter* dan melakukan analisis fungsional dengan menggunakan *testbench* dan *waveform* serta melakukan *static timing analysis* dengan menggunakan Quartus untuk mendapatkan nilai *setup time* dan *hold time*.
- *Instruction Memory* dibutuhkan untuk menyimpan instruksi yang akan dieksekusi, penggunaan *word-indexing* dan *byte-indexing* akan mengeluarkan instruksi yang sama.
- Register x0 digunakan untuk menyimpan nilai nol pada register, hal ini digunakan untuk mempermudah akses pada nilai nol dan membuat operasi menjadi lebih mudah.
- RISC-V menggunakan *instruction set* yang telah ditetapkan yang terdiri dari R-type, I-type, S/B-type, dan J/U-type dan dibedakan dengan menggunakan *opcode, funct3,* dan *funct7*.
- *Control unit* mengatur aliran data dengan melakukan *decoding* instruksi dari *opcode, funct3,* dan *funct7* yang kemudian mengalirkan sinyal pada komponen ALU, Mux, *data memory, register file,* dan *adder*.
- *Instruction set* dari RISC-V memiliki bagian-bagian yang selalu sama posisinya untuk mempermudah melakukan *decoding* instruksi.
- *Data memory* memiliki kapasitas yang lebih besar dibandingkan dengan *register file*, namun untuk akses data dapat dilakukan lebih cepat dengan menggunakan *register file*.
- Pembacaan dilakukan pada *negative edge* untuk menunggu penulisan stabil pada *positive edge* sehingga tidak terjadi tabrakan.
- Tahap-tahap eksekusi instruksi terdiri dari *instruction fetch, instruction decode, execute, data memory,* dan *write back*.
- Pada desain *pipelined*, *throughput* yang dikeluarkan dapat mencapai 5 kali lebih tinggi dibandingkan dengan *single-cycle* pada satu siklus, tetapi dengan *hardware cost* dan *delay* yang lebih tinggi.
- *Decoding instruction set* dilakukan pada *control unit* untuk membaca *opcode, funct7*, dan *funct3*. Kemudian dilakukan *decoding* juga di *register file* untuk membaca rs1, rs2, dan rd.
- *Program Counter* digunakan untuk melakukan iterasi pada memory *instruction file* sehingga seluruh instruksi dapat dikeluarkan.

Referensi

[1] S. L. Harris and D. M. Harris, Digital Design and Computer Architecture, RISC-V Edition. Cambridge, MA, USA: Morgan Kaufmann, 2021.

[2] P. Mittal, ""Timing Paths": Static Timing Analysis (STA) basic (Part 1)," *VLSI Concepts*, Mar. 9, 2011. [Online]. Available: https://www.vlsi-expert.com/2011/03/static-timing-analysis-sta-basic-timing.html. [Accessed: Oct. 30, 2026].

[3] Program Studi Teknik Elektro, Modul Praktikum EL3011 Arsitektur Sistem Komputer. Bandung: Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, 2026.

Lampiran

1. Testbench untuk Tugas 1

```
// Praktikum EL3011 Arsitektur Sistem Komputer
// Modul : 1
// Percobaan : 1
// Tanggal : 28 Oktober 2026
// Nama (NIM) 1 : Danny Rahmat Pratama (13224019)
// Nama (NIM) 2 : Aziiz Adityatama (13224070)
// Nama File : tb_instr_rom_rv32i.v
// Deskripsi : Testbench for Instruction ROM 32x32 Single-Cycle RISC-V (RV32I)
module tb_instr_rom_rv32i;

// Variables for testbench
reg [31:0] ADDR;
reg clock;
reg reset;
wire [31:0] INSTR;
integer i;
integer error;
reg [31:0] mem [0:31];

instr_rom_rv32i u0 (
    .ADDR(ADDR),
    .clock(clock),
    .reset(reset),
    .INSTR(INSTR)
);

always #10 clock = ~clock;

task init;
 begin
    for (i = 0; i < 32; i = i + 1) mem[i] = 32'h00000013;
    mem[0] = 32'h00100293;
    mem[1] = 32'h00000313;
    mem[2] = 32'h00A00393;
    mem[3] = 32'h0053C863;
    mem[4] = 32'h00530333;
    mem[5] = 32'h00128293;
    mem[6] = 32'hFF5FF06F;
    mem[7] = 32'h00000513;

    clock <= 0;
    reset <= 0;
    ADDR <= 0;
    error = 0;
 end
endtask

task testing;
begin
```

```
    for(i = 0; i < 32; i = i + 1) begin
        @(negedge clock);
        reset <= $random;
        ADDR <= i << 2;
        @(negedge clock)
        if(INSTR == mem[i] && reset !== 1'b1) begin
            $display("PASS (address = %h, actual = %h, expected = %h, reset = %h)", ADDR,
INSTR, mem[i], reset);
        end else if (INSTR == 32'h00000013 && reset == 1'b1) begin
            $display("PASS (address = %h, actual = %h, expected = %h, reset = %h)", ADDR,
INSTR, 32'h00000013, reset);
        end else begin
            $display("FAIL (address = %h, actual = %h, expected = %h, reset = %h)", ADDR,
INSTR, mem[i], reset);
            error = error + 1;
        end
    end
end
endtask

initial begin
    $dumpfile("instr_rom_rv32i.vcd");
    $dumpvars(0, tb_instr_rom_rv32i);

    init;
    testing;

    $display("Testbench ended with %d errors", error);
    $finish;
end

 endmodule
```

2. Testbench untuk Tugas 2

```
// Praktikum EL3011 Arsitektur Sistem Komputer
// Modul : 1
// Percobaan : 1
// Tanggal : 28 Oktober 2026
// Nama (NIM) 1 : Danny Rahmat Pratama (13224019)
// Nama (NIM) 2 : Aziiz Adityatama (13224070)
// Nama File : tb_reg_file_rv32i.v
// Deskripsi : Testbench untuk register file RISC-V (RV32I) 32×32
// Timescale for simulation
`timescale 1ns/1ps

module tb_reg_file_rv32i;

// Variables for testbenches
reg             cu_rdwrite;
reg             clock;
reg     [4:0]   rs1_addr;
```

```
reg     [4:0]   rs2_addr;
reg     [4:0]   rd_addr;
reg     [31:0]  rd_in;
wire    [31:0]  rs1;
wire    [31:0]  rs2;

// 32 x Registers (32-bit)
 reg [31:0] rf [0:31];
 integer i;
 integer error;
 reg [31:0] expected[0:9];

// Initialize the clock
always #10 clock = ~clock;

// Taking the main module to the testbench
reg_file_rv32i u0 (
    .clock(clock),
    .cu_rdwrite(cu_rdwrite),
    .rs1_addr(rs1_addr),
    .rs2_addr(rs2_addr),
    .rd_addr(rd_addr),
    .rd_in(rd_in),
    .rs1(rs1),
    .rs2(rs2)
);

task init;
begin
    for (i = 0; i < 32; i = i + 1) begin
        rf[i] = 32'b0;
    end
    cu_rdwrite <= 0;
    clock <= 0;
    rs1_addr <= 0;
    rs2_addr <= 0;
    rd_addr <= 0;
    rd_in <= 0;
    error = 0;
    for (i = 0; i < 10; i = i + 1) begin
        expected[i] = 32'b0;
    end
    expected[1] = 32'h00000011;
    expected[2] = 32'h00000022;
    expected[3] = 32'h00000033;
    expected[4] = 32'h00000044;
end
endtask

task testing;
begin
```

```
    @(negedge clock);
    cu_rdwrite <= 1;

    @(negedge clock);
    rd_addr <= 0;
    rd_in <= 32'hFFFFFFFF;

    for(i = 1; i < 5; i = i + 1) begin
        @(negedge clock);
        rd_addr <= i;
        rd_in <= expected[i];
    end

    @(negedge clock);
    cu_rdwrite <= 0;

    for(i = 0; i < 10; i = i + 1) begin
        @(negedge clock);
        rs1_addr <= i;
        @(negedge clock);
        #1;
        if(rs1 !== expected[i]) begin
            $display("-----------------------------------");
            $display("FAIL");
            $display("T = %0t | alamat  = %h | actual = %h | expected = %h", $time,
rs1_addr, rs1, expected[i]);
            error = error + 1;
        end else begin
            $display("-----------------------------------");
            $display("PASS");
        end
    end
end
endtask

initial begin
    $dumpfile("reg_file_rv32i.vcd");
    $dumpvars(0, tb_reg_file_rv32i);

    init;
    testing;

    #5
    $display("Testbench selesai dengan %d error", error);
    $finish;

end

endmodule
```

3. Testbench untuk Tugas 3

```
// Praktikum EL3011 Arsitektur Sistem Komputer
// Modul : 1
// Percobaan : 1
// Tanggal : 28 Oktober 2026
// Nama (NIM) 1 : Danny Rahmat Pratama (13224019)
// Nama (NIM) 2 : Aziiz Adityatama (13224070)
// Nama File    : tb_ctrl_unit_rv32i.v
// Deskripsi    : Tesbench Control Unit, 3 port input (opcode, funct3, funct7),
//                16 port output/sinyal pengendali
module tb_ctrl_unit_rv32i;

 // Inputs (driven by testbench)
    reg  [6:0] opcode;
    reg  [2:0] funct3;
    reg  [6:0] funct7;

    // Outputs (driven by DUT)
    wire        cu_ALU1src;
    wire        cu_ALU2src;
    wire [2:0]  cu_immtype;
    wire [1:0]  cu_ALUtype;
    wire        cu_adtype;
    wire [1:0]  cu_gatype;
    wire [1:0]  cu_shiftype;
    wire        cu_slttype;
    wire [1:0]  cu_rdtype;
    wire        cu_rdwrite;
    wire [2:0]  cu_loadtype;
    wire        cu_store;
    wire [1:0]  cu_storetype;
    wire        cu_branch;
    wire [2:0]  cu_branchtype;
    wire        cu_jump;

    // Device under test
    ctrl_unit_rv32i u0 (
        .opcode        (opcode),
        .funct3        (funct3),
        .funct7        (funct7),

        .cu_ALU1src    (cu_ALU1src),
        .cu_ALU2src    (cu_ALU2src),
        .cu_immtype    (cu_immtype),
        .cu_ALUtype    (cu_ALUtype),
        .cu_adtype     (cu_adtype),
        .cu_gatype     (cu_gatype),
        .cu_shiftype   (cu_shiftype),
        .cu_slttype    (cu_slttype),
        .cu_rdtype     (cu_rdtype),
```

```
        .cu_rdwrite    (cu_rdwrite),
        .cu_loadtype   (cu_loadtype),
        .cu_store      (cu_store),
        .cu_storetype  (cu_storetype),
        .cu_branch     (cu_branch),
        .cu_branchtype (cu_branchtype),
        .cu_jump       (cu_jump)
    );

task init;
begin
    opcode = 0;
    funct3 = 0;
    funct7 = 0;
end
endtask

task show;
begin
    $display("-----------------------------------------------");
    $display("T=%0t | opcode=%b funct3=%b funct7=%b", $time, opcode, funct3, funct7);
    $display("ALU1src=%b  ALU2src=%b  immtype=%b", cu_ALU1src, cu_ALU2src, cu_immtype);
    $display("ALUtype=%b  adtype=%b  gatype=%b  shiftype=%b  slttype=%b",
              cu_ALUtype, cu_adtype, cu_gatype, cu_shiftype, cu_slttype);
    $display("rdtype=%b  rdwrite=%b  loadtype=%b", cu_rdtype, cu_rdwrite, cu_loadtype);
    $display("store=%b  storetype=%b", cu_store, cu_storetype);
    $display("branch=%b  branchtype=%b  jump=%b", cu_branch, cu_branchtype, cu_jump);
end
endtask

initial begin
    $dumpfile("ctrl_unit_rv32i.vcd");
    $dumpvars(0, tb_ctrl_unit_rv32i);

    #10;
    opcode = 7'b0110011;
    funct7 = 7'b0000000;
    funct3 = 3'b000;
    #5
    show;

    #10;
    opcode = 7'b0010011;
    funct3 = 3'b101;
    #5
    show;

    #10;
    opcode = 7'b0000011;
    funct3 = 3'b010;
```

```
    #5
    show;

    #10;
    opcode = 7'b0100011;
    funct3 = 3'b010;
    #5
    show;

    #10;
    opcode = 7'b1100011;
    funct3 = 3'b001;
    #5
    show;

    #5;
    $finish;
end

endmodule
```

4. Testbench untuk Tugas 4

```
// Praktikum EL3011 Arsitektur Sistem Komputer
// Modul : 1
// Percobaan : 1
// Tanggal : 28 Oktober 2026
// Nama (NIM) 1 : Danny Rahmat Pratama (13224019)
// Nama (NIM) 2 : Aziiz Adityatama (13224070)
// Nama File : data_mem_rv32i.v
// Deskripsi : Testbench Data Memory 256 x 32-bit

module tb_data_mem_rv32i;

reg clock;
reg cu_store;
reg [1:0] cu_storetype;
reg [31:0] dmem_addr;
reg [31:0] rs2;
wire [31:0] dmem_out;
integer i;


data_mem_rv32i u0 (
    .clock(clock),
    .cu_store(cu_store),
    .cu_storetype(cu_storetype),
    .dmem_addr(dmem_addr),
    .rs2(rs2),
```

```
    .dmem_out(dmem_out)
);

always #10 clock = ~clock;

task init;
begin
    clock = 0;
    cu_store = 0;
    cu_storetype = 0;
    dmem_addr = 0;
    rs2 = 0;
end
endtask

task testing;
begin
    @(posedge clock);
    cu_store <= 1;

    for(i = 0; i < 10; i = i + 1) begin
        @(posedge clock);
        rs2 <= $random;
        dmem_addr <= {22'd0, ($random & 32'h000003FF)};
        cu_storetype <= {$random} % 4;
        @(posedge clock);
        case(cu_storetype)
        // SB (Store-Byte)
        2'b00:
        case(dmem_addr[1:0])
            2'b00:
            if (dmem_out[7:0] == rs2[7:0])
                $display("PASS");
            else
                $display("FAIL");

            2'b01:
            if (dmem_out[15:8] == rs2[7:0])
                $display("PASS");
            else
                $display("FAIL");

            2'b10:
            if (dmem_out[23:16] == rs2[7:0])
                $display("PASS");
            else
                $display("FAIL");

            2'b11:
            if (dmem_out[31:24] == rs2[7:0])
```

```
                $display("PASS");
            else
                $display("FAIL");
        endcase

        // SH (Store Halfword)
        2'b01:
        case(dmem_addr[1])
            1'b0:
            if (dmem_out[15:0] == rs2[15:0])
                $display("PASS");
            else
                $display("FAIL");

            1'b1:
            if (dmem_out[31:16] == rs2[15:0])
                $display("PASS");
            else
                $display("FAIL");
        endcase

        // SW (Store Word)
        2'b10:
        if (dmem_out[31:0] == rs2[31:0])
                $display("PASS");
            else
                $display("FAIL");
        endcase
    end
end
endtask

initial begin
    $dumpfile("data_mem_rv32i.vcd");
    $dumpvars(0, tb_data_mem_rv32i);

    init;
    testing;

    #10
    $finish;
end


endmodule
```